Homework Assignment 3

Plan-and-Execute Data Analysis Agent

ECE 157C / ECE 272C
University of California, Santa Barbara
Instructor: Seoyeon Kim

Due: May 15, 2026

1 Overview

In Homework 1, you built an agent that generates and executes Python code to answer questions over a dataset.
In Homework 2, you extended this system to support interaction, memory, and visualization.
In this homework, you will build a **plan-and-execute agent**.
Given a CSV dataset and a natural language question, your system must:

- generate a structured analysis plan,
- execute the plan using predefined table operators,
- produce intermediate results,
- generate a final answer grounded in computed data.

Key Constraint: You are NOT allowed to directly generate Python code to solve the problem. All computation must be expressed as a sequence of table operations.
The goal of this assignment is to separate **reasoning** (planning) from **execution** (computation).

2 Core Principle

All reasoning happens in the plan. Execution is deterministic.

Your system should follow a clear separation of roles:

- **Planner (LLM)**: interprets the natural language question and generates a structured plan
- **Executor**: applies operations step-by-step without any interpretation
- **Answer Generator**: produces the final answer based only on computed results

If your executor needs to interpret meaning or make decisions, your design is incorrect.

3 System Workflow

$$\texttt{Question} \rightarrow \texttt{Plan} \rightarrow \texttt{Execution} \rightarrow \texttt{Result} \rightarrow \texttt{Answer}$$

A correct system should:

- translate the question into a sequence of operations,
- execute those operations deterministically,
- produce reproducible results.

4 Table Operator Tools

You must implement a set of table operators that form the execution layer of your system.

Required Operators

- `derive_columns`
- `filter_rows`
- `group_and_aggregate`
- `sort_rows`
- `limit_rows`
- `select_columns`
- `distinct_rows`

Additional Operators

You are encouraged to implement additional operators if needed. Examples include:

- joins between tables
- handling missing values
- advanced aggregations
- transformations not covered by the required set

Operator Requirements

Each operator must:

- take structured inputs (not natural language),
- perform a well-defined transformation,
- return a new table,
- be deterministic and reproducible.

Operators should behave like standard data processing functions.

5 Plan Representation

The plan defines how the question is solved.

Plan Structure

The plan must follow this format:

```
{
  "steps": [
    { ... },
    { ... }
  ]
}
```

Each step represents one operation applied to the current table.

Step Format

Each step must include:

- `op`: operator name
- operator-specific parameters

Example Plan

```
{
  "steps": [
    {
      "op": "derive_columns",
      "derive": [
        {
          "new_column": "yield",
          "type": "arithmetic",
          "operation": "divide",
          "left": {"type": "column", "value": "pass_count"},
          "right": {"type": "column", "value": "tested_count"}
        }
      ]
    },
    {
      "op": "group_and_aggregate",
      "group_by": ["test_program"],
      "metrics": [
        {
          "function": "mean",
          "column": "yield",
          "as": "avg_yield"
```

```
        }
      ]
    },
    {
      "op": "sort_rows",
      "sort_by": [
        {"column": "avg_yield", "direction": "desc"}
      ]
    },
    {
      "op": "limit_rows",
      "k": 1
    }
  ]
}
```

Execution Model

Execution follows a simple pipeline:

- Step 1 operates on the original dataset
- Each step operates on the result of the previous step
- Steps are executed strictly in order

6 Critical Requirement: No Natural Language in Execution

The executor must NOT interpret natural language.
All logic must be explicitly represented in the plan.

Example

What is the test program with yield lower than 90?

Correct transformation:

```
{
  "op": "filter_rows",
  "conditions": [
    {
      "column": "yield",
      "operator": "<",
      "value": 0.9
    }
  ]
}
```

Important: Vague expressions must be resolved into numeric conditions.
If execution requires interpreting words like “high”, “low”, or “large”, the plan is incorrect.

7 Dataset Requirement

You must select a **new dataset**.

- Must NOT reuse datasets from previous assignments
- 100 – 50,000 rows
- At least 5 columns

Choose a dataset that supports multi-step analysis.

8 Question Design

Create:

- 5 basic questions
- 5 intermediate questions
- 5 advanced questions

Requirement

All questions must require multiple operations.
Examples of required complexity:

- filtering + aggregation
- derived column + grouping
- sorting + top-k selection

Single-step questions receive zero credit.

9 Evaluation Requirements

For each question, you must provide:

- question
- generated plan
- execution trace
- final answer

Execution Trace

You must show how the table changes across steps.

```
Step | Operation           | Input Rows | Output Rows
-----------------------------------------------------
1    | derive_columns      | 10000      | 10000
2    | filter_rows         | 10000      | 3200
3    | group_and_aggregate | 3200       | 10
```

10 Report Requirements

Your `Report.pdf` should clearly document both your system design and your experimental results. The goal of the report is not just to show that your system works, but to demonstrate your understanding of:

- how planning differs from execution,
- how structured operators affect system behavior,
- the trade-offs between different agent designs.

1. Dataset Description

Describe the dataset you selected:

- source of the dataset
- number of rows and columns
- description of key columns
- why this dataset requires multi-step analysis

2. Question Design

List all 15 questions (5 basic, 5 intermediate, 5 advanced).
For each question:

- explain why it requires multiple steps
- describe the expected sequence of operations
- explain why it is classified as basic / intermediate / advanced

3. System Design

Describe your system architecture:

- how the planner generates the plan
- how the executor applies each operation
- how operators are implemented
- how intermediate results are stored and passed between steps

4. Execution Analysis

Select at least 3 representative questions and provide full analysis.
For each question, include:

- the natural language question
- the generated plan
- the execution trace (step-by-step table changes)
- the final answer

Then explain:

- why the plan is correct (or incorrect)
- how each step contributes to the final result

5. Failure Analysis (Very Important)

Include at least 2 failure cases.
Each failure must include:

- the question
- the generated plan
- what went wrong
- whether the issue is:
  - planning error
  - operator limitation
  - execution issue
- how you fixed or improved it

6. Comparison with Previous Agents

Compare your current system with the agents from previous assignments.
Discuss:

- differences between:
  - code-generation agents (HW1)
  - interactive/memory-based agents (HW2)
  - plan-and-execute agents (HW3)
- advantages of structured plans and operators
- limitations compared to generating arbitrary code

- trade-offs such as:
  - flexibility vs reliability
  - expressiveness vs controllability
  - automation vs interpretability

7. Reflection

Provide your overall insights:

- What types of questions are easy or difficult for your system?
- What are the limitations of your current operator set?
- What additional operators would improve your system?
- How would you extend this system in the future?

11 Project Structure

You must organize your submission using the following structure:

```
project/
|-- agent.py
|-- planner.py
|-- executor.py
|-- operators.py
|-- test_agent.py
|-- dataset.csv
|-- results.csv
|-- Report.pdf
```

File Descriptions

- `agent.py`
  Main entry point of your system. It should take a user question and dataset path, run the full pipeline (plan → execute → answer), and return the result.
- `planner.py`
  Responsible for generating the structured plan from a natural language question using an LLM.
- `executor.py`
  Executes the generated plan step-by-step using the operator functions.
- `operators.py`
  Contains implementations of all required table operators (e.g., `derive_columns`, `filter_rows`, etc.). You may also include any additional operators you define.

- `test_agent.py`
  Script used to test your system on multiple questions. It should demonstrate that your agent works end-to-end.
- `dataset.csv`
  The dataset you selected for this assignment.
- `results.csv`
  A file containing your results for all questions.
- `Report.pdf`
  Your report documenting system design, experiments, and analysis.

Results File Format

Your `results.csv` must follow this format:

```
question, plan, final_answer
```

Each row should correspond to one question.

12 Grading

- Plan correctness: 30%
- Operator implementation: 25%
- Execution correctness: 20%
- Question quality: 15%
- Report: 10%

Deductions

- trivial questions
- missing execution trace
- use of natural language in execution

13 Final Advice

Reflect on how this agent design compares to your previous approaches. In your report, discuss the key trade-offs and design choices.